ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<5>>

Descripción

```python
def req_5(catalog, n, pais, fecha_inicio, fecha_fin):
    """
    Retorna el resultado del requerimiento 5
    """
    start_time = get_time()

    pedidos_pais = msc.get(catalog["OrdenPaisSC"], pais)
    productos = msc.new_map(100, 1)
    lista_productos = lt.new_list()

    total_pedidos = 0
    recaudo_total = 0

    if pedidos_pais is not None:
        for i in range(lt.size(pedidos_pais)):
            pedido = lt.get_element(pedidos_pais, i)
            fecha = pedido["Order_Date"]

            if fecha_inicio <= fecha <= fecha_fin:
                total_pedidos += 1

                nombre = pedido["Product"]
                amount = pedido["Amount"]
                cajas = pedido["Boxes_Shipped"]
                precio = pedido["Price_per_Box"]
                descuento = pedido["Discount_Pct"]
                marketing = pedido["Marketing_Spend"]

                recaudo_total += amount

                producto = msc.get(productos, nombre)

                if producto is None:
                    producto = {
                        "Product": nombre,
                        "numero_pedidos": 0,
                        "total_cajas": 0,
                        "recaudo_total": 0,
                        "suma_precios": 0,
                        "suma_descuentos": 0,
                        "marketing_total": 0
                    }

                    msc.put(productos, nombre, producto)
                    lt.add_last(lista_productos, producto)

                producto["numero_pedidos"] += 1
                producto["total_cajas"] += cajas
                producto["recaudo_total"] += amount
                producto["suma_precios"] += precio
                producto["suma_descuentos"] += descuento
                producto["marketing_total"] += marketing

    total_productos = lt.size(lista_productos)
```

```
    for i in range(total_productos):
        producto = lt.get_element(lista_productos, i)
        cantidad = producto["numero_pedidos"]

        producto["precio_promedio"] = producto["suma_precios"] / cantidad
        producto["descuento_promedio"] = producto["suma_descuentos"] / cantidad

    def comparar(producto1, producto2):
        recaudo1 = producto1["recaudo_total"]
        recaudo2 = producto2["recaudo_total"]

        if recaudo1 != recaudo2:
            return recaudo1 > recaudo2

        cajas1 = producto1["total_cajas"]
        cajas2 = producto2["total_cajas"]

        if cajas1 != cajas2:
            return cajas1 > cajas2

        return producto1["Product"] <= producto2["Product"]

    lt.merge_sort(lista_productos, comparar)

    top_productos = lt.new_list()

    cantidad_mostrar = n

    if cantidad_mostrar > total_productos:
        cantidad_mostrar = total_productos

    if cantidad_mostrar < 0:
        cantidad_mostrar = 0

    for i in range(cantidad_mostrar):
        producto = lt.get_element(lista_productos, i)

        resumen = {
            "Product": producto["Product"],
            "numero_pedidos": producto["numero_pedidos"],
            "total_cajas": producto["total_cajas"],
            "recaudo_total": producto["recaudo_total"],
            "precio_promedio": producto["precio_promedio"],
            "descuento_promedio": producto["descuento_promedio"],
            "marketing_total": producto["marketing_total"]
        }

        lt.add_last(top_productos, resumen)

    end_time = get_time()
    tiempo = delta_time(start_time, end_time)

    return {
        "tiempo": tiempo,
        "total_pedidos": total_pedidos,
        "total_productos": total_productos,
        "recaudo_total": recaudo_total,
        "top_productos": top_productos
    }
```

Identifica los N productos con mayor recaudo para un país dentro de un rango inclusivo de fechas. Para esto, se consulta la tabla hash OrdenPaisSC (Separate Chaining), construida durante la carga, cuya llave es el país y cuyo valor es un array_list con sus pedidos. Esto permite acceder directamente a los registros del país seleccionado.

Después se recorren esos pedidos, se filtran por fecha y se agrupan por producto mediante otra tabla hash de Separate Chaining llamada productos. En esta tabla, la llave es el nombre del producto y el valor es un diccionario donde se acumulan la cantidad de pedidos, cajas enviadas, recaudo, precios por caja, descuentos y gasto de marketing.

Finalmente, se calculan los promedios y se ordenan los productos utilizando Merge Sort por recaudo descendente, cajas descendentes y nombre alfabético ascendente. Se presentan los primeros N productos del ranking.

Entrada	catalog, n, pais, fecha_inicio, fecha_fin
Salidas	tiempo, total_pedidos, total_productos, recaudo_total, top_productos (con número de pedidos, cajas, recaudo, precio y descuento promedio y marketing total de cada producto).
Implementado (Sí/No)	Implementado por el grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Buscar el país en OrdenPaisSC mediante msc.get (Separate Chaining)	O(1) promedio
Recorrer los p pedidos del país y comprobar el rango de fechas	O(p)
Agrupar y acumular datos mediante la tabla hash productos	O(1) promedio por pedido
Calcular promedios de los d productos diferentes	O(d)
Ordenar los d productos con Merge Sort	O(d log d)
Construir la lista de los primeros min(N, d) productos	O(min(N,d))
TOTAL	***O(p + d log d)***

Análisis

Al utilizar OrdenPaisSC, podemos acceder directamente a los pedidos del país solicitado, con un costo de búsqueda O(1) promedio. Esto nos evita recorrer todos los registros del archivo y limita el procesamiento a los p pedidos de ese país.

La tabla hash auxiliar productos también nos permite acumular las estadísticas de cada producto en tiempo O(1) promedio por operación. Después, el ordenamiento con Merge Sort tiene un costo O(d log d), donde d es la cantidad de productos diferentes encontrados. Por lo tanto, la complejidad temporal promedio de nuestro requerimiento es O(p + d log d).

La construcción del índice por país tiene complejidad O(n) promedio durante la carga, mientras que la carga completa alcanza O(n log n) por el ordenamiento general. Como estas operaciones se realizan antes de la consulta, no se incluyen en su tiempo de ejecución. La ventaja de nuestra solución es que agrupamos los pedidos mediante hash y ordenamos únicamente los productos diferentes, lo que reduce el trabajo necesario para obtener el Top N.